

How to Run the Screener on Your Computer

How to run the UW app on your computer

Don't want to install anything? Use the cloud version — open app-compliance-screener-tihxaacrkagcy2ijutdvsk.streamlit.app in your browser and start screening immediately.

Quick start (2 steps)

1. **Open Terminal** (Mac: Spotlight → “Terminal”; Windows: Command Prompt or PowerShell).
2. **Go to the project folder and start the app:**

```
cd "/Users/tetianamedvid/Documents/Base44 analysis"  
python3 run_uw_app.py
```

Sync runs first, then the dashboard. **Open your browser** at **http://localhost:8501**. (Wait a few seconds if the page doesn't load.)

Important: While the app is running, the **terminal will not give you a new prompt** — it will look “stuck.” That is normal. The app is running. Use your browser to open <http://localhost:8501>. When you want to stop the app, go back to the Terminal and press **Ctrl+C**.

Recommended: Run `python3 run_uw_app.py` — it runs full sync (validate, fetch profiles, UW for missing) then starts the dashboard; you only open the browser. To skip sync and start the dashboard only, set `SKIP_STARTUP_SYNC=1` in `.env` or run `python3 -m streamlit run streamlit_uw.py --server.port 8501`.

Run QA (resolve + full profile): From the project folder run `python3 scripts/qa_uw_lookup.py`.

Debug Trino conversation: Run `python3 scripts/debug_trino_conversation.py <app_id>` to see conversation history fetched directly from Trino (no local data). See `docs/DEBUG-TRINO-CONVERSATION.md`.

Trino setup and troubleshooting: Configuration, connection test, 403 Forbidden, and local-only mode. See docs/TRINO-SETUP-AND-TROUBLESHOOTING.md. It exits 0 if lookup and profile work for the test app; non-zero if something is wrong (e.g. merge order or missing data).

Check dashboard is ready to QA: Run python3 scripts/qa_dashboard_ready.py. It checks that the app list loads (e.g. 99 apps), a sample app has profile rows, and prints the run command. Then start the app and QA in the browser.

Dashboard QA checklist (in browser): (1) App list shows 99 apps in the expander. (2) Pick an App ID (e.g. from the list), paste it, click **Look up** — App profile, Conversation history, and UW verdict (if run) appear. (3) Open **Debug: data source** to confirm Trino/local and full profile loaded. (4) For an app without a verdict, **Run UW for this app** should create a conclusion and show the verdict after reload.

No API keys: The app does not use any API keys. It uses either local JSON (sample or custom file) or Trino (host only, no key in .env).

Full data, unlimited (as designed and QA'd)

The app is designed for **full profile data** and **unlimited** lookups. Two ways to get that:

Mode	How	Result
Trino (live)	Set TRINO_HOST in .env to your Trino coordinator host. Run the app; log in with SSO if prompted.	Full data for any app (look up by App ID, MSID, or WixPayments account ID). Unlimited.
Local (full export)	Put a single JSON file in the project with full records for every app. Each record must include at least: app_id, app_name, app_url, msid, account_id, conversation_summary. You can include any other fields (e.g. entity_id, base44_email, date_updated) — they will all appear in the profile. Set APPS_JSON_PATH in .env to that file (e.g. APPS_JSON_PATH=data/my_full_export.json). Restart the app.	Full data for every app in that file . No limit on how many apps; each gets full profile.

Full profile when Trino is unavailable: Use a JSON export that includes first_activity_at, user_description, user_app_events_count, etc., and set APPS_JSON_PATH; the app will show those fields from the file instead of from Trino.

Extended profile + conversation from a separate file: You can add data/full_profiles.json (or set FULL_PROFILES_JSON_PATH in .env) — an object keyed by app_id, each value having extended profile fields and optionally conversation_snapshots (list of {created_at, content}) or conversation_summary. The app uses this when Trino is off so profile and conversation history are still filled.

JSON format (full record): Same as data/production_test_apps.json: array of objects. Each object should have app_id, app_name, app_url, msid, account_id, conversation_summary. Any extra fields are shown in the app profile.

You don't have to manually export every time. Set APPS_JSON_PATH in .env **once** to the path of your JSON file. If that file is updated automatically (e.g. by a scheduled export from Quix, a script, or a shared folder), open the app and click  **Reload app list** to use the latest data. No restart, no manual export each time.

UW check vs policy: Verdict and reasoning come from **cached pipeline results** (output/run_*/conclusion_<app_id>.md). Run the underwriting pipeline for an app to generate a verdict; then the app will show it.

Underwriting pipeline: LLM options (no OpenAI key required)

You **do not need** OPENAI_API_KEY in .env to run the pipeline. The **7-app sample** in this repo was run **without** an OpenAI key using **Ollama** (local LLM); see output/run_production_test_ollama/.

Option	How	When to use
Ollama (local)	Install Ollama , run ollama pull llama3.1:8b, then run the pipeline (default --llm auto tries OpenAI then Ollama). No API key.	You want real LLM verdicts and have Ollama on your machine.
Template only	Run with --llm none. No API key, no Ollama.	You only need conclusion files with a safe “Restricted / manual review” style verdict.
OpenAI	Set OPENAI_API_KEY in .env and run with default -- llm auto or -- llm openai.	You prefer OpenAI and have a key.

Example (no key, template-only):

```
python3 run_underwriting.py --apps data/app_tribeology_one.json --  
policy policy/policy-excerpt.txt --out output --run-id  
my_run --llm none --no-scrape
```

Example (Ollama, no key):

```
# Terminal 1: ollama serve (if not already running)  
# Terminal 2:  
python3 run_underwriting.py --apps data/real_apps.json --policy  
policy/policy-excerpt.txt --out output --run-id  
ollama_run --llm ollama
```

Faster batch runs — Use `--workers 8` to process apps in parallel (scraping and LLM calls run concurrently). Use `--delay 0` to skip the 0.5s pause after each LLM call (faster but may hit rate limits). Use `--no-scrape` when `conversation_summary` + `user_description` already give enough signal.

Save app summary (middleman) — The pipeline caches the middleman app summary in `output/uw_cache/` by `app_id`. On re-runs, cached summaries are reused so the middleman step is skipped (only policy comparison runs). Use `--no-cache` to force regeneration.

```
python3 run_underwriting.py --apps data/real_apps.json --policy  
policy/policy-excerpt.txt --out output --run-id fast_run  
--workers 8 --delay 0
```

Why not DORA / MCP? MCP (e.g. Trino MCP, DORA) in Cursor is for the **AI assistant** (Cursor) to call tools while you chat. The underwriting pipeline runs as a **standalone script** (`run_underwriting.py`); it does not run inside Cursor, so it cannot use Cursor's MCP connections. To use a DORA-backed LLM from the pipeline you would add a small backend in the repo (e.g. `_call_dora()` in `run_underwriting.py`) that calls whatever **HTTP/API** DORA exposes for programmatic use (URL + auth in `.env`). If your team provides a DORA completion API, that can be wired in the same way as OpenAI/Ollama.

Real data extract (not the 7 QA samples)

The 7 apps in `data/production_test_apps.json` are for QA. For **real data** you need your own extract.

Step 1 — Get a real export

- From **Quix**: run your Trino (or SQL) query, then **export the results** to CSV or JSON (use Quix's export / download option if available).
- Or from your **data team / DB**: get a CSV or JSON export of apps with at least columns like: `app_id`, `app_name`, `app_url`, `msid`, `account_id`, `conversation_summary` (column names can vary; the script maps common names).

Step 2 — Run the import script

From the **project folder** in Terminal:

```
cd "/Users/tetianamedvid/Documents/Base44 analysis"
python3 scripts/import_apps_export.py path/to/your_export.csv
```

Use your actual file path. For JSON:

```
python3 scripts/import_apps_export.py path/to/your_export.json
```

The script writes **data/real_apps.json** (or use `--out data/other_name.json` to choose the file name).

Step 3 — Point the app at the real list

1. Open **.env** in the project.
2. Set (or change) this line:
 APPS_JSON_PATH=data/real_apps.json
3. Save the file.
4. **Restart the app** (Ctrl+C in the terminal, then `python3 run_uw_app.py`).

After that, the app uses your **real data extract** and shows full profiles for every app in that file.

Automatic refresh (no manual steps) — If the export is dropped in a fixed place (e.g. by Quix, a cron job, or a shared folder), you can have the app **re-import that file on its own**:

1. In **.env** set **APPS_REFRESH_SOURCE_PATH** to the path of that file (e.g. **APPS_REFRESH_SOURCE_PATH=data/exports/latest_apps.csv**). Use the same path you would pass to `import_apps_export.py`.
2. Keep **APPS_JSON_PATH** pointing at the file the app should read (e.g. `data/real_apps.json`). The refresh will overwrite that file from the source.
3. Optional: **APPS_REFRESH_EVERY_SECONDS=3600** (default: 1 hour). The app will re-import at most once per hour, and once when you open or reopen the app (whichever comes first).
4. No dialogs, no popups — refresh runs in the background. The UI can show “Data last refreshed: ...” under the local-data caption. Click **Reload app list** after a refresh to see the latest data without restarting.

So: point the source path at where the export lands; the app keeps the list updated every hour or on open, without you running the script or clicking export each time.

QA only: no scripts, no .env — Open the app, expand “ **Update app list from export**”, upload your CSV or JSON export. The app imports it and uses it as the app list. Then use **Reload app list** if needed and QA. The only step the app *cannot* do is run the Trino query (Quix/Trino returns 403 for programmatic access); you run the query in Quix and export once, then everything else is in the app.

Use local data only (works immediately — no Trino needed)

If you **don't have a working Trino host** (e.g. you only have Quix UI access) or want the app to work **right away** without any connection:

1. **Open** `.env` in the project folder.
2. **Add this line** (or uncomment it if you see it):
`TRINO_USE_LOCAL_ONLY=1`
3. **Save** the file (Cmd+S / Ctrl+S).
4. **Run the app** (Quick start above) and open **`http://localhost:8501`**.

The app will use **only your local app list** (no Trino, no 403 errors). You can look up any app that's in `data/production_test_apps.json` or in a custom file. To **add more apps**: put a JSON file in the project (same format as `production_test_apps.json`: each object has `app_id`, `app_name`, `app_url`, `msid`, `account_id`, `conversation_summary`) and in `.env` set e.g. `APPS_JSON_PATH=data/my_apps.json`. Restart the app.

Summary: Set `TRINO_USE_LOCAL_ONLY=1` in `.env` → run the app → use the app list. No Trino host required.

No Trino, no API keys: use local JSON only

If you don't have a Trino host and don't want to use any API keys:

- **Default:** The app uses the built-in list in `data/production_test_apps.json` (7 sample apps). Run the app and use one of the App IDs from the expander.
 - **More apps:** Put a JSON file in the project (same format as `production_test_apps.json`: each object has `app_id`, `app_name`, `app_url`, `msid`, `account_id`, `conversation_summary`). In `.env` add one line, e.g. `APPS_JSON_PATH=data/my_apps.json`. Restart the app. No keys, no Trino.
-

Enable Trino live with SSO (no API keys)

Step-by-step for non-devs. No API keys — you only add the Trino host; login is done in the browser with SSO.

Step 1: Open the `.env` file

1. Open **Cursor** (or any text editor).
2. In the **left sidebar**, click your project folder (e.g. **Base44 analysis**).
3. Click the file `.env` to open it. (If you don't see it: **File** → **Open File** and choose `.env` in the project folder.)

What you see: The file opens with lines starting with `TRINO_...`

Step 2: Add your Trino host

1. Find the line **TRINO_HOST=** or **TRINO_DEFAULT_HOST=** (may be empty after =).
2. Use **TRINO_DEFAULT_HOST=** if everyone uses the same host (admin sets once). Use **TRINO_HOST=** for just you.
3. **After the =**, type the Trino host, no spaces. Example:
TRINO_DEFAULT_HOST=trino.mycompany.com or
TRINO_HOST=trino.mycompany.com
4. **Save: Cmd+S** (Mac) or **Ctrl+S** (Windows).

What you see: File saved.

Step 3: Run the app

1. Open **Terminal** (Mac: Spotlight → **Terminal**; Windows: **Command Prompt** or **PowerShell**).
2. Type: `cd "/Users/tetianamedvid/Documents/Base44 analysis"` and press **Enter**. (Change the path if your folder is elsewhere.)
3. Type: `python3 run_uw_app.py` and press **Enter**.
4. Wait a few seconds. The terminal will look “stuck” (no new line). That is normal.

What you see: The app is starting.

Step 4: Open the app in your browser

1. Open your **web browser**.
2. In the **address bar** type: `http://localhost:8501` and press **Enter**.

What you see: The UW Lookup page opens. You may see “Connecting to Trino (SSO)...”.

Step 5: Log in with SSO if a browser window opens

1. If **another browser window or tab** opens asking you to log in, that is the **SSO** page for Trino.
2. **Log in** the way you usually do at your company.
3. Go back to the **UW Lookup** tab. **Refresh** the page (F5) if it still says “Connecting...”.

What you see: The page should say “Using live data from Trino (SSO)”. You can look up any app.

Step 6: Stop the app when you’re done

1. Go back to the **Terminal** window.
2. Press **Ctrl+C** (hold Control and press C).

What you see: The app stops. The browser tab won’t work until you run the app again (Step 3).

Summary: Add the Trino host in `.env` (Step 2). Run the app (Step 3) and open `http://localhost:8501` (Step 4). If a login window opens (Step 5), log in with SSO; then the app uses live Trino with no API keys. If you don’t set a Trino host, the app uses local data only.

Production: look up any app (fully functional)

The project already has a **.env** file. To look up **any** app (not just the sample list):

1. **Open the file .env** in the project folder (in Cursor: click it in the file list, or File → Open → choose **.env**).
2. **Find the line** that says `TRINO_HOST=`.
3. **After the =**, type your Trino host. (If you don't have it, ask your platform or data team; it looks like `trino.yourcompany.com`.) Example:
`TRINO_HOST=trino.yourcompany.com`
4. **Save** the file (Cmd+S).
5. **Connect to VPN** if your company requires it to reach Trino.
6. **Start the app** (Quick start above). It will show **“Using live data from Trino”** and you can look up any app.

Until you set `TRINO_HOST`, the app uses sample data and only the listed test app IDs work. Restart the app after you change **.env**.

How to do production (step-by-step)

Follow these steps in order.

Step 1 — Get the Trino host

- Ask your platform, data, or infra team: *“What is the Trino host for the analytics / data cluster?”*
- They will give you a hostname, e.g. `trino.company.com` or `trino.internal.company.com`. Write it down or copy it.

Step 2 — Put it in .env

1. In **Cursor**, in the left sidebar (file list), find the file **.env** in your project folder. (If you don't see it, use **File** → **Open File** and choose **.env** in the project folder.)
2. **Click** **.env** to open it.
3. Find the line that says: `TRINO_HOST=`
4. **After the =**, paste the host you got (no spaces). Example:
`TRINO_HOST=trino.company.com`
5. **Save** the file: press **Cmd+S** (Mac) or **Ctrl+S** (Windows).

Step 3 — Connect to VPN

- If your company uses VPN to access internal tools, **connect to VPN** the way you usually do. (If no one has told you to use VPN for data tools, you can skip this and see if the app works.)

Step 4 — Start the app

1. Open **Terminal** (Mac: Spotlight → “Terminal”; Windows: Command Prompt or PowerShell).
2. Go to the project folder:


```
cd "/Users/tetianamedvid/Documents/Base44 analysis"
```

3. Run:

```
python3 run_uw_app.py
```

4. You will see one or two lines of text, then the terminal will **stay busy** (no new prompt). That is normal — the app is running.

Step 5 — Open the app in your browser

1. Open your **web browser** (Chrome, Safari, Firefox, etc.).
2. In the address bar type: **http://localhost:8501**
3. Press **Enter**. The UW Lookup page should open. Under the search box it should say **“Using live data from Trino”** if `.env` is set correctly.

Step 6 — Stop the app when you’re done

- Go back to the **Terminal** window where the app is running.
- Press **Ctrl+C** (hold Control and press C). The app stops and the terminal will show a prompt again.

If you don’t have a team to ask for the host: Ask your manager or a colleague who works with data or analytics: “Who can give me the Trino host for querying our data?” or “What is the hostname for our Trino / analytics cluster?”

Wix / Quix: “403 Forbidden” means wrong host

If the app shows **“403 Forbidden”** and mentions **Wix.com** or **bo.wix.com**, you are using the **Quix web UI address** as `TRINO_HOST`. That URL opens the data platform in your browser; it is **not** the Trino API the app needs.

- **Do not use:** `bo.wix.com` (or the URL you use to open Quix in the browser) as `TRINO_HOST`.
- **Do:** Get the **Trino coordinator host** from Quix or your data platform docs — e.g. a host like `trino...` or a dedicated “Trino connection” / “JDBC/ODBC endpoint” host. Put that in `.env` as `TRINO_HOST=...`

Until you set the correct Trino host, the app will keep using sample/local JSON data.

Verify: Run `python3 scripts/test_live_trino.py` — it prints the exact Trino error when connection fails. See `docs/TRINO-SETUP-AND-TROUBLESHOOTING.md` for full details.

Use your browser session so the app runs Trino as you

If **you** can run Trino queries in Quix in the browser, the app can use the same session so it runs as you.

1. **Log into Quix** in your browser (open `bo.wix.com`, go to Quix, run a query so you're logged in).
2. **Copy your session cookie:**
 - Open **Developer Tools**: in Chrome use the menu **View** → **Developer** → **Developer Tools**; or **right-click** anywhere on the page → **Inspect**. (You don't need to use the F12 key.)
 - In the panel that opens, click the **Application** tab (Chrome) or **Storage** (Firefox).
 - In the **left sidebar**, expand **Cookies**, then click `https://bo.wix.com`.
 - In the table (Name, Value, ...), find a session cookie (e.g. **Name** = `session` or `sid`). **Double-click** the **Value** cell and copy it (Cmd+C or Ctrl+C).
 - **If you don't see "Application"**: the panel has a row of tabs (Elements, Console, Network, Application, ...). Click **Application**. Then in the left sidebar click the ► next to **Cookies** and choose `bo.wix.com`.
3. **Paste into .env:**

Add one line (use the cookie name and value you copied):
`TRINO_COOKIE=session=THE_VALUE_YOU_COPIED`
(or the full cookie string, e.g. `name1=value1; name2=value2`).
Use a **single line**, no line breaks.
If you have a **Bearer token** instead, use:
`TRINO_AUTH_HEADER=Bearer YOUR_TOKEN`
4. **Use Trino from the app:**

In `.env` leave `TRINO_HOST=bo.wix.com` and **remove or comment out** the line `TRINO_USE_LOCAL_ONLY=1` (so the app tries Trino).
Restart the app and open `http://localhost:8501`. The app will send your cookie/token with each request so it runs Trino as you.

Note: Cookies and tokens expire. If the app starts getting 403 again, repeat steps 1–3 to refresh the value in `.env`.

What you need before you start

- The **project folder** on your computer (the folder that contains `uw_app`, `data`, `output`, and files like `run_underwriting.py`).
 - **Python 3** installed. (If you've run the underwriting pipeline before, you already have it.)
-

Part 1: Run the app (sample data)

Step 1: Open Terminal

- **On Mac:** Open the **Terminal** app (search for "Terminal" in Spotlight).

- **On Windows:** Open **Command Prompt** or **PowerShell** (search in the Start menu).

What you see: A window with a blinking cursor where you can type.

Step 2: Go to the project folder

1. Type this (change the path if your project folder is somewhere else):

```
cd "/Users/tetianamedvid/Documents/Base44 analysis"
```

2. Press **Enter**.

What you see: The line of text at the top of the window (the “prompt”) may change to show the folder name. You are now “in” the project folder.

Step 3: Install what the app needs (only once)

1. Type:

```
pip3 install -r requirements.txt
```

2. Press **Enter**.

3. Wait until it finishes (you may see a lot of text; that’s normal).

What you see: At the end it usually says something like “Successfully installed ...”. You only need to do this step once (or again if someone tells you dependencies changed).

For full scraping of JavaScript apps (e.g. base44.app): After pip install, run once: `python3 -m playwright install chromium` This downloads the browser used to render JavaScript pages. **Without this step, scraped content will be minimal** (e.g. “You need to enable JavaScript to run this app” instead of the real page content). Run it once per machine.

Scraping architecture (Base44-aware): The scraper uses different strategies depending on the URL:

- **base44.app URLs (React SPAs):** Skips plain HTTP fetch (always returns JS shell). Goes directly: Base44 public API → SEO meta tags → Playwright rendering. This saves 2-8 seconds per app.
- **Custom domain URLs:** If the HTML contains Base44 markers, automatically probes Base44 public APIs too.
- **Non-Base44 URLs:** Uses fast HTTP fetch first, falls back to Playwright only if needed.

What the scraper extracts: - **SEO meta tags** (title, description, og:image) from raw HTML — available even before JS renders - **Base44 public API data** — app name, description, visibility (public/private), login methods, payment signals, integrations, AI agents - **Entity data** (data model) — captures entity names, record counts, field names, and sample data from SDK API calls during rendering - **Backend function**

calls — detects function names like `createCheckout`, `processPayment` which reveal app capabilities - **Auth wall classification** — distinguishes between private apps (by design), public apps with auth, and broken apps - **Sitemap pages** — probes `sitemap.xml` for page structure - **Product extraction** — JSON-LD and DOM-based product/price extraction

Step 4: Start the app

1. Type (this shows all output in the Terminal):

```
python3 -m streamlit run streamlit_uw.py --server.port 8501
```

2. Press **Enter**. Wait a few seconds.

What you see: A line saying “Starting UW Lookup app ...”, then more text. **Wait 5–10 seconds** for more lines to appear. A browser tab may open by itself. **If no tab opens**, open your browser yourself and go to: **`http://localhost:8501`**.

If the app doesn’t start at all:

1. **Look at the Terminal window.** Scroll up and see if there is any **red text** or a line that says **Error** or **Traceback**. That message tells you what went wrong.
2. **Try opening the app by hand.** In your browser’s address bar type: **`http://localhost:8501`** and press Enter. Sometimes the app is running but the browser didn’t open automatically.
3. **If you see “Operation not permitted” or “Address already in use”:** Another program may be using the same port, or a firewall may be blocking it. Try closing any other Streamlit or Python app, then run `python3 run_uw_app.py` again. If it still fails, copy the **full error message** from the Terminal and share it with your team so they can suggest a fix (e.g. using a different port).

If you see one of these errors:

- **“streamlit: command not found”** — Install dependencies first (Step 3). Then use: `python3 run_uw_app.py` again.
- **“ImportError: attempted relative import with no known parent package”** — One file in the project needs a small fix. Do this once:
 1. **Where to edit:** In Finder, open your project folder (e.g. **Base44 analysis**). Inside it, open the folder **uw_app**. Open the file **streamlit_app.py** (in Cursor or any text editor — double-click or right-click → Open With → Cursor/TextEdit).
 2. **What to change:** Select and delete everything from the **very first line** of that file down to (and including) the line that contains **`trino_configured`**. The next line left in the file should be the blank line just before `st.set_page_config`.
 3. **Paste this at the top:** At the top of the file, paste exactly this (copy the whole block):

```
"""
```

```
UW Internal App – desktop version. Run on your laptop (no SS0).
```

```
python3 -m streamlit run streamlit_uw.py --server.port
8501
or: python3 run_uw_app.py
"""
import sys
from pathlib import Path

# Ensure project root is on path so "uw_app" can be
imported when run as a script.
_project_root = Path(__file__).resolve().parent.parent
if str(_project_root) not in sys.path:
    sys.path.insert(0, str(_project_root))

import json
import streamlit as st
from uw_app.resolve import resolve, DEFAULT_APPS_JSON
from uw_app.uw_cache import get_uw_for_app
from uw_app.profile import profile_from_app_record,
profile_from_trino_row
from uw_app.trino_client import get_full_profile,
is_configured as trino_configured
```

4. **Save** the file (e.g. Cmd+S). Then go back to **Terminal**, make sure you're still in the project folder (if in doubt, type `cd "/Users/tetianamedvid/Documents/Base44 analysis"` and press Enter), and run again: `python3 run_uw_app.py`.

Step 5: Use the app in the browser

1. You'll see a page titled **UW Lookup**.
2. Under **"I have"**, choose what you're looking up: **App ID (24-character)**, **MSID (UUID)**, or **WixPayments account ID (UUID)**.
3. In the box next to it, **paste or type** the value (for example an app ID like 698406273ade17b9bd851188).
4. Click the red **Look up** button.

Standalone URL check: At the bottom, paste any app URL and click **Scrape & Run UW**. Use **Analysis: Auto** to get LLM verdicts when OpenAI or Ollama is available; use **Deep scrape** to capture API responses, follow links, and extract products. See "Underwriting pipeline: LLM options" above for setup.

What you see: The page shows **App profile** (a table of fields and values) and **UW check vs policy** (verdict, reasoning, and expandable sections). If you're on sample data, only certain app IDs work; the page tells you and shows which ones.

Step 6: Stop the app when you're done

1. Go back to the **Terminal** window where the app is running.
2. Press **Ctrl+C** (hold Control and press C).

What you see: The app stops. The browser tab will no longer work until you run Step 4 again.

Part 2: Use live Trino (look up any app)

By default the app uses **sample data** (only a few test apps). To look up **any** app, you need to use **live Trino**. Your team has access to Trino over VPN and SSO. Follow these steps.

Step 1: Copy the example config file

1. In **Terminal**, make sure you're in the project folder (see Part 1, Step 2).
2. Type:

```
cp .env.example .env
```

3. Press **Enter**.

What you see: Nothing visible; a new file named `.env` is created in the project folder.

Step 2: Fill in the Trino host

1. Open the **project folder** in Finder (Mac) or File Explorer (Windows).
2. Find the file named **.env** (it may be hidden; you might need to show hidden files).
3. **Open .env** with a text editor (e.g. TextEdit, Notepad).
4. Find the line that says `TRINO_HOST=`.
5. After the `=`, type your **Trino host** (your internal docs or platform team will give you this; it looks like a web address, e.g. `trino.mycompany.com`).
6. Save the file and close it.

What you see: The line might look like: `TRINO_HOST=trino.mycompany.com`. The other lines (`TRINO_PORT`, `TRINO_CATALOG`, `TRINO_USER`) can stay as they are unless your team says otherwise.

Step 3: Connect to VPN (and SSO if required)

1. Connect to your company **VPN** the way you usually do.
2. If your organization uses **SSO** (single sign-on) to access data tools, sign in as required.

What you see: You're connected; your computer can now reach Trino.

Step 4: Start the app (same as Part 1, Step 4)

1. In **Terminal**, in the project folder, type:

```
python3 -m streamlit run streamlit_uw.py --server.port 8501
```

2. Press **Enter**.

What you see: The browser opens. On the UW Lookup page, under the search box it should say “**Using live data from Trino**”. If it still says “Using sample data”, check that `.env` has `TRINO_HOST=` filled in and that you’re on VPN.

Step 5: Look up any app

1. Choose **App ID**, **MSID**, or **WixPayments account ID** under “I have”.
2. Paste or type the value.
3. Click **Look up**.

What you see: App profile and UW check for that app (if a conclusion exists for it in the pipeline output).

Part 3: Check that live Trino works (optional)

You can test the connection to Trino without opening the full app.

Step 1: Be on VPN and have `.env` set

1. Make sure you’ve done Part 2, Steps 1–3 (copy `.env`, set `TRINO_HOST`, connect to VPN).
-

Step 2: Run the test script

1. In **Terminal**, go to the project folder (Part 1, Step 2).
2. Type:

```
python3 scripts/test_live_trino.py
```

3. Press **Enter**.

What you see:

- If it works: it prints “Resolve: OK”, “Full profile: OK”, and “Live Trino test passed.”
 - If it fails: it may say “TRINO_HOST is not set” (then check your `.env` file) or an error about connection (then check VPN and the host value).
-

Short version (after first-time install)

If you've already run Part 1 once and only want to start the app:

1. Open **Terminal**.
2. Type: `cd "/Users/tetianamedvid/Documents/Base44 analysis"` and press **Enter**.
3. Type: `python3 -m streamlit run streamlit_uw.py --server.port 8501` and press **Enter**.
4. Use the browser at **<http://localhost:8501>**.

That's it.